

development or for learning purposes.

Kubernetes can also be run on IoT platforms like Raspberry Pi for IoT applications and for low-cost projects.

The Kubernetes cluster has multiple components such as:

- Kubernetes Proxy—for routing network traffic for load balancing services (<https://kubernetes.io/docs/getting-started-guides/scratch/>)
- Kubernetes DNS—a DNS server for naming and discovery of the services that are defined in DNS
- Kubernetes UI—this is the GUI to manage the cluster

Pods in Kubernetes

A pod is a collection of application containers and volumes running in the same execution environment. One can say that the pods are the smallest deployable artifacts in the Kubernetes cluster environment. Every container within a pod runs in its own cgroup but shares a number of Linux name spaces. Pods can be created using the following command in the CLI:

```
kubectl run kuard
```

Most of the pod manifests are written using YAML or JSON scripts. But YAML is preferred as it is in human readable format.

There are various command options using `kubectl` to run or list pods.

Labels, annotations and service discovery

Labels are key-value pairs that can be attached to Kubernetes objects such as pods and replica-sets. These labels help in finding the required information about Kubernetes objects, metadata for objects and for the grouping of objects.

Annotations provide a place to store additional metadata for Kubernetes objects with assisting tools and libraries.

Labels and annotations go hand-in-hand; however, annotations are used to provide extra information about where and how an object came from and what its policies are.

Table 2: A comparison between daemon sets and replica sets

Daemon sets	Replica sets
Ensures a copy of the pod is running across a set of nodes in the Kubernetes cluster.	These are generally about creating a service with multiple replicas for redundancy purposes.
These are used to deploy system daemons such as log collectors and monitoring agents. Generally, these should be used when we want a single copy of our application to run on all nodes or a subset of them in the cluster.	These should be used when the application we are using is completely decoupled from the node in the cluster.

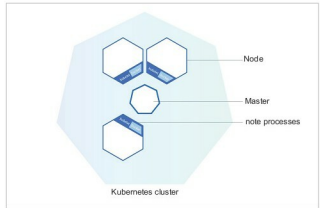


Figure 3: Kubernetes cluster (<https://kubernetes.io/docs/tutorials/kubernetes-basics/deploy-intro/>)

A comparison between Docker Swarm and Kubernetes

Both Kubernetes and Docker Swarm are popular and used as container orchestration platforms. Docker also started supporting and shipping Kubernetes from its CE (community edition) and EE (enterprise edition) releases.

Docker Swarm is the native clustering for Docker. Originally, it did not provide much by way of container automation, but with the latest update to Docker Engine 1.12, container orchestration is now built into its core with first-party support.

It takes some effort to get Kubernetes installed and running, as compared to the faster and easier Docker Swarm installation. Both have good scalability and high availability features built into them. Hence, one has to choose the right one based on the need of the hour. For more details, do refer to <https://www.upcloud.com/blog/docker-swarm-vs-kubernetes/>. 

References

- [1] <https://kubernetes.io/docs/getting-started-guides/scratch/>
- [2] <https://kubernetes.io/docs/setup/independent/create-cluster-kubeadm/>
- [3] <https://docs.docker.com/engine/docker-overview/#docker-architecture>
- [4] <https://kubernetes.io/docs/tutorials/kubernetes-basics/deploy-intro/>
- [5] <https://www.upcloud.com/blog/docker-swarm-vs-kubernetes/>

By: Shashidhar Soppin

The author is a senior architect and has 16+ years of experience in the IT industry—in virtualisation, the cloud, Docker, open source software, ML, deep learning and OpenStack. He is part of the product engineering team at Wipro. You can reach him at shashi.soppin@gmail.com.